

Aula 04 - Procedural consolidado com funções por índice

Contexto de negócio

Fechamento diário: consolidar fluxo procedural em funções para reduzir erro operacional.

Com arrays paralelos, como manter regras de negócio e exibição sem espalhar lógica no `main`?

Tensão operacional

Sem funções dedicadas, o `main` vira mistura de regra e exibição.

Objetivo técnico da entrega

- Separação de funções de negócio (`depositar`, `sacar`) e exibição (`mostrarConta`).
- Confirmação de resultado de operação com boolean.

Recurso técnico desta aula

Recurso inserido: `mostrarConta`; os dados permanecem separados por arrays.

Problema que justifica o novo artefato

Mesmo com arrays, o main pode virar um bloco confuso misturando regra e saída. Sem separação de responsabilidades, manutenção e teste ficam caros.

Artefato explicado: função de exibição e rotina operacional

Artefatos desta aula:

- `mostrarConta(...)` para separar apresentação;
- funções de negócio (`depositar`, `sacar`) mantendo foco em regra.

O algoritmo fica particionado em passos claros: processar, validar resultado e exibir estado.

Transição didática para a próxima aula

Com fluxo consolidado, aparece um debate de design: contrato de retorno boolean versus retorno de estado para saque.

Valor prático entregue

- Melhora rastreabilidade do fluxo de negócio.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.

- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execução em Java.
public class AppGestaoContas {

    static void depositar(double[] p_saldos, int p_indice, double p_valor) {
        p_saldos[p_indice] = p_saldos[p_indice] + p_valor;
    }

    static boolean sacar(double[] p_saldos, double[] p_limites, boolean[]
p_ativas, int p_indice, double p_valor) {
        if (!p_ativas[p_indice]) {
            return false;
        }

        double saldoDisponivel = p_saldos[p_indice] + p_limites[p_indice];
        if (p_valor > 0 && p_valor <= saldoDisponivel) {
            p_saldos[p_indice] = p_saldos[p_indice] - p_valor;
            return true;
        }
        return false;
    }

    static void mostrarConta(String[] p_titulares, double[] p_saldos, int
p_indice) {
        System.out.println("Titular: " + p_titulares[p_indice] + " | Saldo: " +
p_saldos[p_indice]);
    }

    public static void main(String[] args) {
        // Aula 04: procedural consolidado com funções de negócio e exibição.
        String[] titulares = {"Ana", "Bruno"};
        double[] saldos = {1000.00, 300.00};
        double[] limites = {500.00, 100.00};
        boolean[] ativas = {true, true};

        depositar(saldos, 0, 150.00);
        boolean saqueBruno = sacar(saldos, limites, ativas, 1, 250.00);

        System.out.println("Saque do Bruno aprovado? " + saqueBruno);
        mostrarConta(titulares, saldos, 0);
        mostrarConta(titulares, saldos, 1);
    }
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Discussão de contrato de retorno

Quando o método retorna `boolean`, o contrato precisa ser tratado como regra de negócio:

- `true` significa operação aprovada e estado atualizado;
- `false` significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o `false` no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (`true` e `false`) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários `boolean` basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o `false` em produção?

Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não “conhece” a conta por contexto implícito.
- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;
- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Na POO, estado e comportamento passam a coexistir no objeto.

Perguntas de decisão

1. O que melhorou ao separar exibição da regra?
2. O que ainda continua frágil no modelo?
3. Qual próximo passo natural para reduzir parâmetros e índices?

Variações de cenário

1. Tentar saque acima do limite e observar retorno.
2. Chamar `mostrarConta` antes e depois da operação.

Exercícios progressivos

1. Criar `mostrarResumoGeral` percorrendo todos os índices.
2. Bloquear saque quando `ativas[indice]` for `false`.

Desafio de equipe

Adicionar função `aplicarTarifaMensal(double[] saldos, double tarifa)` para todas as contas ativas.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.

Checklist de progressividade técnica

- Pré-requisito confirmado: leitura e execução do código-base da aula anterior.
- Novidade técnica identificada: explicar em uma frase o recurso novo desta aula.
- Complexidade controlada: apontar qual decisão de projeto evita acoplamento desnecessário.
- Evidência prática: executar ao menos um caso de sucesso e um caso de erro no Tryit.

Fechamento executivo

Chegamos ao limite útil do procedural inicial: volume e regras estão funcionando, mas dados ainda estão espalhados. Esse problema prepara a virada para classe na etapa seguinte da trilha.

Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?
2. Risco real: qual erro operacional essa implementação reduz?
3. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler

2. Colar todo o conteúdo de AppGestaoContas.java no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_methods.asp
- https://www.w3schools.com/java/java_if_else.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>